LeetCode HOT 100

PART 6

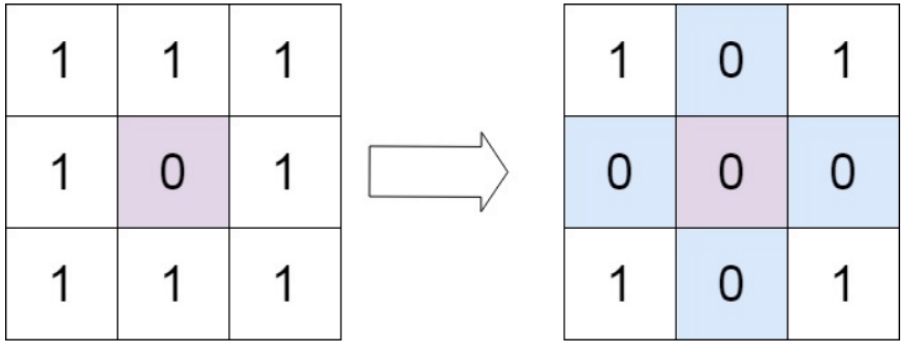
Matrix

矩阵

18. 矩阵置零 (No.73) MEDIUM

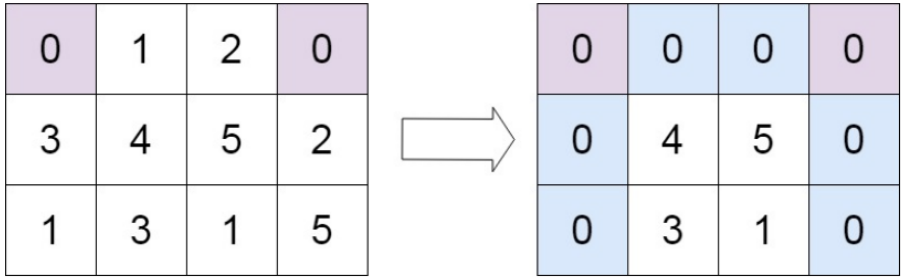
给定一个 `m x n` 的矩阵，如果一个元素为 **0** ，则将其所在行和列的所有元素都设为 **0** 。请使用 **原地** 算法。

示例 1：



```
输入：matrix = [[1,1,1],[1,0,1],[1,1,1]]
输出：[[1,0,1],[0,0,0],[1,0,1]]
```

示例 2：



```
输入：matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]
输出：[[0,0,0,0],[0,4,5,0],[0,3,1,0]]
```

提示：

- `m == matrix.length`
- `n == matrix[0].length`
- `1 <= m, n <= 200`
- -2^{31} `<= matrix[i][j] <=` $2^{31} - 1$

进阶：

- 一个直观的解决方案是使用 `O(mn)` 的额外空间，但这并不是一个好的解决方案。
- 一个简单的改进方案是使用 `O(m + n)` 的额外空间，但这仍然不是最好的解决方案。
- 你能想出一个仅使用常量空间的解决方案吗？

18. 矩阵置零 (No.73) MEDIUM

Solution 1 – 标记数组

思路

用row和col记录某行某列是否有0，用一次遍历标记0，再用一次遍历置0.

```
class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        """
        Do not return anything, modify matrix in-place instead.
        """
        m, n = len(matrix), len(matrix[0])
        row, col = [0] * m, [0] * n
        for i in range(m):  # 标记0
            for j in range(n):
                if matrix[i][j] == 0:
                    row[i] = col[j] = 1
        for i in range(m):  # 置0
            for j in range(n):
                if row[i] or col[j]:
                    matrix[i][j] = 0
```

时间复杂度：$\mathcal{O}(mn)$；空间复杂度：$\mathcal{O}(m+n)$

18. 矩阵置零 (No.73) MEDIUM

Solution 2 – 原地标记数组

思路

为了实现$\mathcal{O}(1)$的空间复杂度，用第1行和第1列来存储其他行列是否有0的信息，然后单独用row和col来记录第1行和第1列的信息。

和Solution1不同，这里要用0代表需要置0，否则会改变第1行和第1列的原始信息而无法复原。然后标记0要先处理第1行/列，置0要最后处理第1行/列。

这道题还有个只需要一个变量的官方题解，已经被评论区喷完全是屎山换空间（我个人也觉得没啥意义），Solution2的思想还是要学习一下。

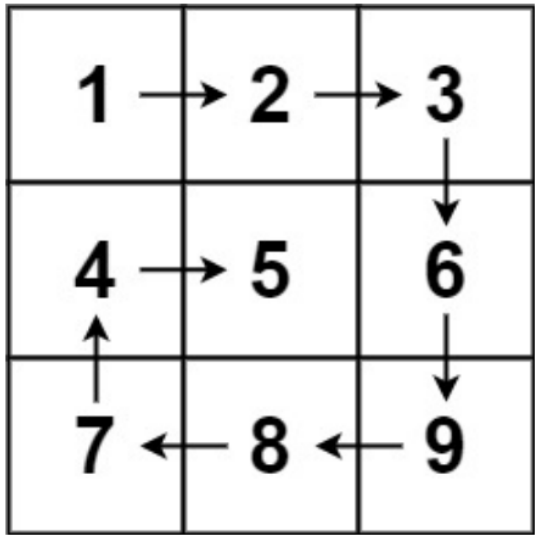
时间复杂度：$\mathcal{O}(mn)$；空间复杂度：$\mathcal{O}(1)$

```python
class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        """
        Do not return anything, modify matrix in-place instead.
        """
        m, n = len(matrix), len(matrix[0])
        row, col = 1, 1  # 记录第1行第1列的情况，0表示要置0
        # 第一行第一列标记0
        for i in range(m):
            if matrix[i][0] == 0:
                col = 0
        for j in range(n):
            if matrix[0][j] == 0:
                row = 0
        # 其余行列标记0，和Solution1不同，这里必须标0表示行列要置0
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] == 0:
                    matrix[i][0] = 0
                    matrix[0][j] = 0
        # 置0要单独最后处理第一行第一列
        for i in range(1, m):
            for j in range(1, n):
                if not matrix[i][0] or not matrix[0][j]:
                    matrix[i][j] = 0
        if not row:  # 第一行
            matrix[0] = [0] * n
        if not col:  # 第一列
            for i in range(m):
                matrix[i][0] = 0
```

19. 螺旋矩阵 (No.54) MEDIUM

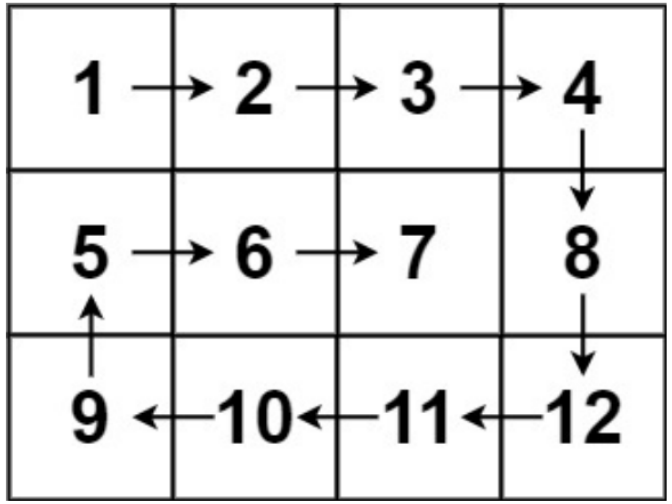
给你一个 `m` 行 `n` 列的矩阵 `matrix` ，请按照 **顺时针螺旋顺序** ，返回矩阵中的所有元素。

示例 1：



输入： `matrix = [[1,2,3],[4,5,6],[7,8,9]]`
输出： `[1,2,3,6,9,8,7,4,5]`

示例 2：



输入： `matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]`
输出： `[1,2,3,4,8,12,11,10,9,5,6,7]`

提示：

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 10`
- `-100 <= matrix[i][j] <= 100`

19. 螺旋矩阵 (No.54) MEDIUM

思路

定义上下左右四个边界，每轮结束时边界-1，直到左边界大于右边界且上边界大于下边界，说明已经缩成一个点走完了。

特别注意：

(1)每轮走完一半要判断一下，如果退化成单行或单列要直接break，否则以单行为例，算法会向右走再向左走重复；

(2)每个方向的遍历要搞清边界，顶点的地方不要重复遍历了

```
class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        m, n = len(matrix), len(matrix[0])
        # 定义各边界
        left, right, top, bottom = 0, n-1, 0, m-1
        i, j = 0, 0
        res = []
        while left <= right and top <= bottom:
            for j in range(left, right + 1):
                res.append(matrix[top][j])
            for i in range(top+1, bottom + 1):
                res.append(matrix[i][right])
            # 如果退化成单行或单列直接退出，否则会重复
            if left >= right or top >= bottom:
                break
            for j in range(right-1, left, -1):
                res.append(matrix[bottom][j])
            for i in range(bottom, top, -1):
                res.append(matrix[i][left])
            left, right, top, bottom = left+1, right-1, top+1, bottom-1
        return res
```

时间复杂度：$\mathcal{O}(mn)$；空间复杂度：$\mathcal{O}(1)$

20. 旋转图像 (No.48) MEDIUM

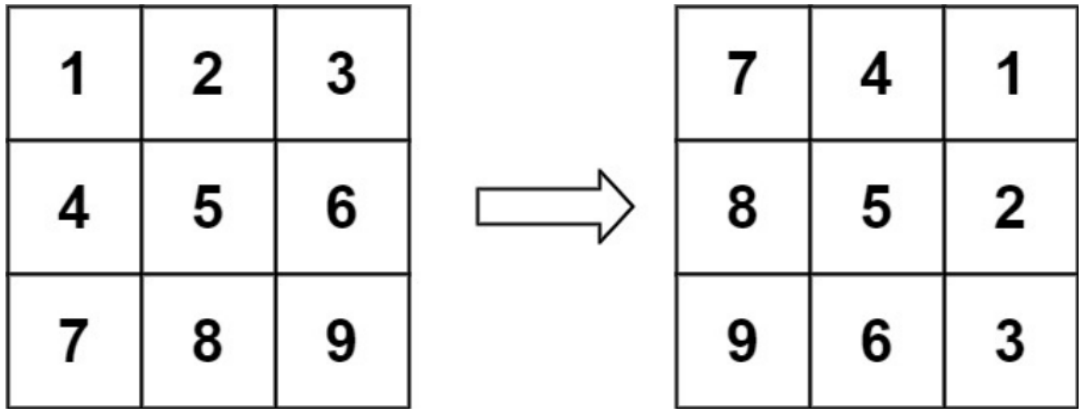
给定一个 $n \times n$ 的二维矩阵 `matrix` 表示一个图像。请你将图像顺时针旋转 90 度。

你必须在 **原地** 旋转图像，这意味着你需要直接修改输入的二维矩阵。**请不要** 使用另一个矩阵来旋转图像。

提示：

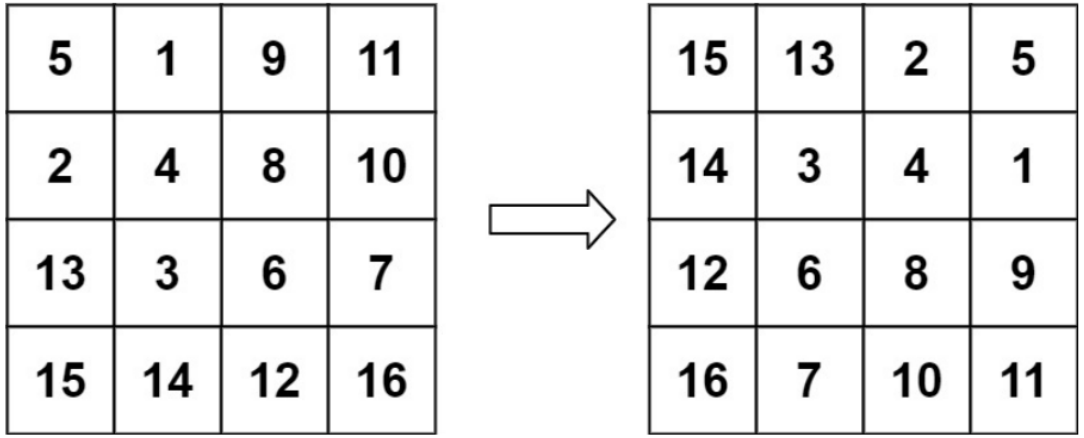
- `n == matrix.length == matrix[i].length`
- `1 <= n <= 20`
- `-1000 <= matrix[i][j] <= 1000`

示例 1：



输入： `matrix = [[1,2,3],[4,5,6],[7,8,9]]`
输出： `[[7,4,1],[8,5,2],[9,6,3]]`

示例 2：



输入： `matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]`
输出： `[[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]`

20. 旋转图像 (No.48) MEDIUM

Solution 1 – 分组原地旋转

思路

方阵旋转是以四个元素为一组轮换的，因此找到所有的这些组，依次进行4次组内换位。组别的数量跟矩阵阶数有关，奇数和偶数阶要分别处理。

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        """
        Do not return anything, modify matrix in-place instead.
        """
        n = len(matrix)
        m = n // 2 + 1 if n % 2 else n // 2
        # 所有元素四个一组替换，只需要遍历左上角的块
        for i in range(m):
            l = m - 1 if n % 2 else m  # 奇数阶矩阵第m行或第m列仅能遍历其一
            for j in range(l):
                tmp, matrix[j][n-i-1] = matrix[j][n-i-1], matrix[i][j]
                tmp, matrix[n-i-1][n-j-1] = matrix[n-i-1][n-j-1], tmp
                tmp, matrix[n-j-1][i] = matrix[n-j-1][i], tmp
                matrix[i][j] = tmp
```

时间复杂度：$\mathcal{O}(n^2)$；空间复杂度：$\mathcal{O}(1)$

20. 旋转图像 (No.48) MEDIUM

Solution 2 – 转置翻转

思路

方阵顺时针旋转90°等价于先转置再进行水平翻转（或先水平翻转再转置）；如果是逆时针旋转等价于先转置再进行垂直翻转（或先垂直翻转再转置）。

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        """
        Do not return anything, modify matrix in-place instead.
        """
        n = len(matrix)
        # 转置
        for i in range(n):
            for j in range(i):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
        # 水平翻转
        for i in range(n):
            for j in range(n//2):
                matrix[i][j], matrix[i][n-j-1] = matrix[i][n-j-1], matrix[i][j]
```

时间复杂度：$\mathcal{O}(n^2)$；空间复杂度：$\mathcal{O}(1)$

21. 搜索二维矩阵 II (No.240) MEDIUM

编写一个高效的算法来搜索 `m x n` 矩阵 `matrix` 中的一个目标值 `target` 。该矩阵具有以下特性：

- 每行的元素从左到右升序排列。
- 每列的元素从上到下升序排列。

示例 1：

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

```
输入：matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],
[18,21,23,26,30]], target = 5
输出：true
```

示例 2：

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

```
输入：matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],
[18,21,23,26,30]], target = 20
输出：false
```

提示：

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= n, m <= 300`
- -10^9 `<= matrix[i][j] <=` 10^9
- 每行的所有元素从左到右升序排列
- 每列的所有元素从上到下升序排列
- -10^9 `<= target <=` 10^9

21. 搜索二维矩阵 II (No.240) MEDIUM

Solution 1 – 二分查找

思路

对矩阵逐行（或逐列）进行二分查找。

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        # 对第i行进行二分查找
        def binary_search(i):
            left, right = 0, n - 1
            while left <= right:
                mid = (left + right) // 2
                if matrix[i][mid] == target:
                    return True
                elif matrix[i][mid] > target:
                    right = mid - 1
                else:
                    left = mid + 1
            return False
        for i in range(m):
            if binary_search(i):
                return True
        return False
```

时间复杂度： $\mathcal{O}(m\log n)$或$\mathcal{O}(n\log m)$；**空间复杂度：** $\mathcal{O}(1)$

21. 搜索二维矩阵 II (No.240) MEDIUM

Solution 2 – 排除法

思路

从右上角开始，target更大就往下找，target更小往左找，超过边界就是False。方法成立的原因在于，每次决定往下找，就排除了左上角的所有点（左上角必然比当前点小）；每次决定往左找，就排除了所有右下角的点（右下角必然比当前点大），因此从右上角开始找，绝不会漏掉可能存在的点。

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        i, j = 0, n-1  # 初始在右上角
        while 0 <= i < m and 0 <= j < n:
            if matrix[i][j] < target:
                i += 1  # 想找更大的往下找
            elif matrix[i][j] > target:
                j -= 1  # 想找更小的往左找
            else:
                return True
        return False
```

时间复杂度：$\mathcal{O}(m+n)$；**空间复杂度**：$\mathcal{O}(1)$